

Image Processing & Computer Vision

Block 2 - Object Detection

Part I - Theory

1. From Classification to Object Detection

To understand object detection, it is helpful to compare it with simpler visual recognition tasks. In image classification, the goal is to assign a single label to the entire image:

$$f : \text{Image} \rightarrow \text{Class Label}$$

For example, a classifier may predict whether an image contains a cat, a car, or a tool. However, this approach does not provide information about the location of the object within the image. Object localization extends this task by predicting both the class label and the position of a single object:

$$f : \text{Image} \rightarrow (\text{Class}, \text{Bounding Box})$$

In many real-world scenarios, images contain multiple objects. Object detection therefore generalizes the problem further by predicting several objects simultaneously:

$$f : \text{Image} \rightarrow \{(\text{Class}_i, \text{BBox}_i)\}_{i=1}^N$$

Each detected object is represented by a class label and a bounding box that indicates its position in the image.

2. Bounding Boxes

Objects in detection tasks are typically represented using rectangular bounding boxes. A bounding box defines the spatial extent of an object within an image.

One common representation uses the center position and size of the box:

- center coordinates (x, y)
- width w
- height h

Another common representation specifies the coordinates of the top-left and bottom-right corners:

- (x_{min}, y_{min})
- (x_{max}, y_{max})

These representations allow neural networks to predict object positions as continuous values.

Bounding boxes provide a simple and computationally efficient way to represent object locations, although they are only an approximation of the true object shape.

3. Challenges in Object Detection

Object detection is significantly more complex than image classification. One difficulty is that images may contain multiple objects belonging to different classes. The detector must therefore identify several objects simultaneously.

Another challenge is scale variation. Objects may appear very small or very large depending on their distance from the camera. Additionally, objects may be partially occluded by other objects or appear in cluttered scenes.

Modern object detectors often predict multiple overlapping bounding boxes for the same object. To remove duplicate detections, a post-processing step called Non-Maximum Suppression (NMS) is applied.

Finally, many applications require real-time processing. For example, detection systems used in robotics, autonomous vehicles, or industrial automation must process images very quickly. These challenges have motivated the development of specialized deep learning architectures for detection.

4. Detector Architectures

Modern object detectors can be divided into two main categories:

Two-Stage Detectors

Two-stage detectors first generate candidate regions that might contain objects. In a second step, each region is classified and its bounding box refined.

The R-CNN family (including Fast R-CNN and Faster R-CNN) follows this approach. Two-stage detectors often achieve very high accuracy, but they are generally slower because multiple processing steps are required.

One-Stage Detectors

One-stage detectors perform object detection in a single forward pass through the network.

A well-known example is YOLO (You Only Look Once). Instead of generating region proposals, YOLO directly predicts bounding boxes and class probabilities for the entire image.

Because of their simpler architecture, one-stage detectors are typically much faster and can operate in real time.

5. The YOLO Detection Concept

The YOLO architecture divides an image into a grid of cells. Each grid cell is responsible for predicting objects whose centers lie within that cell.

For each cell, the network predicts:

- bounding box coordinates
- an objectness score indicating whether an object is present
- class probabilities

During training, the network learns to jointly optimize localization and classification.

The training loss typically includes three components:

- localization loss for bounding box coordinates
- confidence loss for object presence

- classification loss for predicted class labels

This unified formulation allows YOLO models to perform detection very efficiently.

6. Intersection over Union (IoU)

To evaluate how well a predicted bounding box matches the ground truth, the metric *Intersection over Union* (IoU) is commonly used.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

IoU measures the overlap between the predicted bounding box and the ground truth box. A value of 1 indicates a perfect match, while a value of 0 indicates no overlap. Detection predictions are usually considered correct if the IoU exceeds a certain threshold (commonly 0.5).

7. Evaluation Metrics

Object detection performance is typically evaluated using precision and recall. Precision measures how many predicted detections are correct:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall measures how many true objects were successfully detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

A common summary metric is the Average Precision (AP), which measures the area under the precision-recall curve. The mean Average Precision (mAP) is obtained by averaging AP values across all object classes. One widely used benchmark metric is:

$$\text{mAP@0.5}$$

which evaluates detections using an IoU threshold of 0.5.

8. Annotation Formats and Datasets

Object detection models require labeled data for training. These labels specify both the object class and its location within an image. Over time, several standard annotation formats have emerged.

Annotation Formats

The **COCO format** (Common Objects in Context) is one of the most widely used standards. Annotations are stored in a JSON file and include:

- image metadata (file name, size, ID)
- object categories
- bounding boxes (in (x_{min}, y_{min}, w, h) format)
- segmentation masks (optional)

COCO also supports multiple objects per image and additional information. It is commonly used in modern deep learning frameworks.

The **Pascal VOC format** stores annotations in XML files, with one file per image. Each file contains:

- image information (size, filename)
- object labels
- bounding boxes (in $(x_{min}, y_{min}, x_{max}, y_{max})$ format)

This format is simpler and human-readable, making it useful for smaller projects and educational purposes.

The **YOLO format** is designed for efficiency and is widely used with one-stage detectors. Each image has a corresponding text file where each line represents one object:

- class ID
- normalized center coordinates (x, y)
- normalized width w and height h

All values are normalized to the range $[0, 1]$, which simplifies training and improves numerical stability.

Popular Datasets

The **COCO dataset** is one of the largest and most commonly used benchmarks for object detection. It contains:

- over 200,000 labeled images with 80 object categories
- complex scenes with multiple objects per image

It is widely used for training and evaluating modern detection models.

The **Pascal VOC dataset** (Visual Object Classes) is a classic benchmark in object detection. It includes:

- 11,500 images with 20 object categories
- high-quality annotations
- standardized evaluation protocols

Although smaller than COCO, it is still commonly used for benchmarking and teaching.

The **Open Images dataset**, released by Google, is a large-scale dataset containing:

- millions of images
- hundreds of object classes
- bounding boxes, segmentation masks, and relationships

It is useful for training models on a wide variety of real-world scenarios.

Part II - Practical Task

Goal of the Exercise

In this exercise you will build a small object detection system using your own dataset. Instead of using existing benchmark datasets, you will collect images yourself and annotate objects using bounding boxes. The goal is to understand the complete workflow of object detection systems: data collection, annotation, training, and evaluation.

Step 1 - Define Object Classes

Select between three and five object classes. The objects should be easy to capture in different scenes. Possible examples include:

- office objects
- small toys
- grocery items
- tools or mechanical components

Choose objects that can appear together in a single scene.

Step 2 - Capture Scene Images

Capture images containing multiple objects. Try to create realistic scenes that contain variation in object arrangement and appearance. Recommended variations include:

- different object positions
- partial occlusions
- different distances
- different lighting conditions
- different backgrounds

Requirements:

- at least 60 images in total
- at least 10 instances per object class

Step 3 - Annotate Bounding Boxes

Use a labeling tool such as:

- LabelImg
- CVAT
- Roboflow
- MakeSense.ai

For each visible object in an image:

- draw a bounding box
- assign the correct class label

Careful annotation is essential for training reliable detection models.

Step 4 - Train a Detection Model

Train a pretrained detection model such as YOLO on your dataset. Minimum requirements:

- use a pretrained YOLO model (e.g. YOLOv8n)
- train on the annotated dataset

Optional improvements: apply data augmentation; try a larger model variant; tune confidence thresholds.

Step 5 - Evaluate the Model

After training, evaluate the performance of your detection model. Possible evaluation aspects include:

- visualize predictions on unseen images
- identify and interpret false positives and false negatives
- observe how well the model detects small or partially occluded objects
- test the model on images that were not used during training

Discuss typical failure cases and possible reasons for them.

Optional Experiment - Counting Objects with the Detector

In this experiment you will investigate how well an object detector can count a large number of instances of the same object class.

Procedure:

- Select one object class from your dataset.
- Create a set of images containing many instances of this object.
- Arrange the objects in scenes with increasing numbers of instances (for example 5, 10, 15, or more objects).
- Capture images where the objects are:
 - well separated
 - close together
 - partially overlapping
- Run the trained detector on these images.
- Count how many objects the detector predicts in each image.